

UNIT IV: Schema Refinement (Normalization)

- Purpose of Normalization or schema refinement, concept of functional dependency, normal forms based on functional dependency Lossless join and dependency preserving decomposition, (1NF, 2NF and 3NF), concept of surrogate key, Boyce-Codd normal form(BCNF), MVD (Multi Valued Dependency), Fourth normal form(4NF), Fifth Normal Form (5NF).

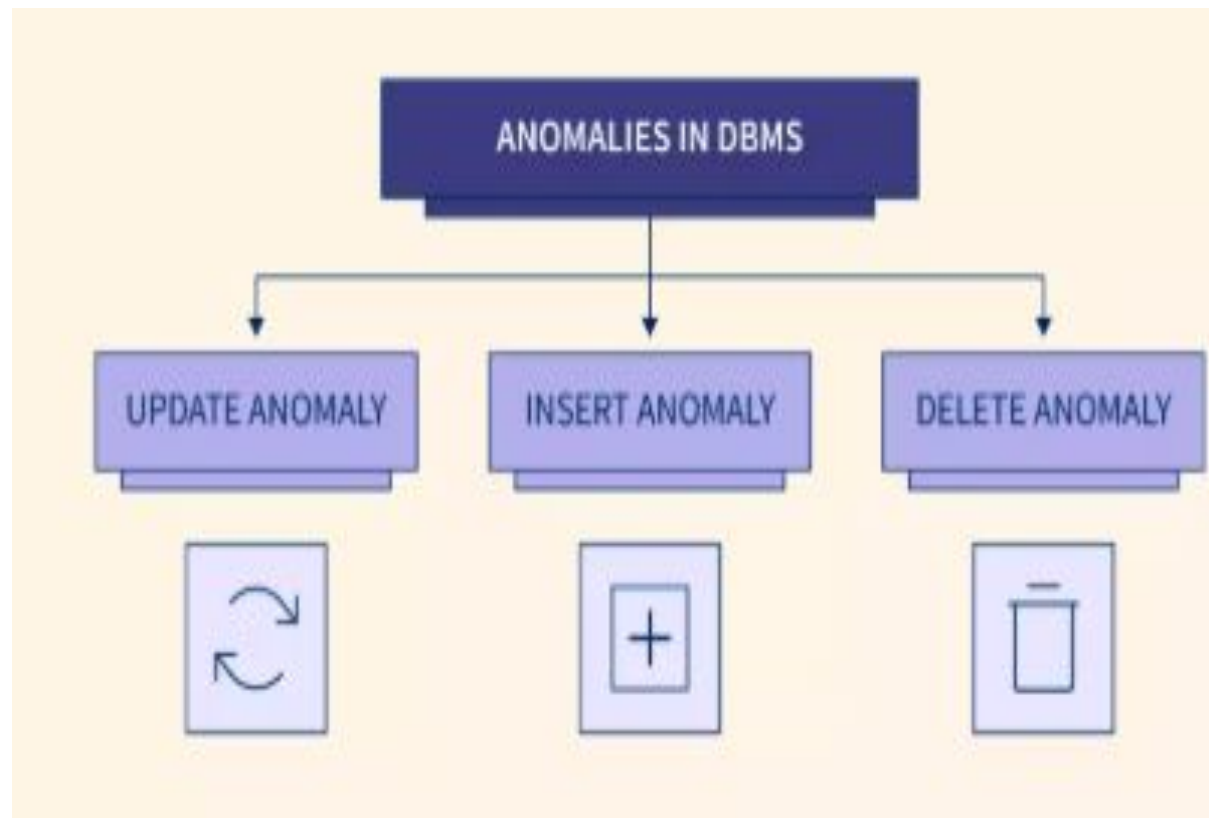
UNIT IV: Schema Refinement (Normalization)

- **Purpose of Normalization or schema refinement,** concept of functional dependency, normal forms based on functional dependency Lossless join and dependency preserving decomposition, (1NF, 2NF and 3 NF), concept of surrogate key, Boyce-Codd normal form(BCNF), MVD, Fourth normal form(4NF), Fifth Normal Form (5NF).

Purpose of Normalization or schema refinement

- It prevents anomalies like **data being inconsistent across multiple tables during inserts, updates, and deletions**, making the database more organized, efficient, flexible, and easier to maintain.
- Storing the same information redundantly, within a database, can lead to several problems:
 - ▣ *Redundant storage*
 - ▣ *Update anomalies*
 - ▣ *Insertion anomalies*
 - ▣ *Deletion anomalies*

Purpose of Normalization or schema refinement



Purpose of Normalization or schema refinement

- **Insert anomaly:** An **Insert Anomaly** occurs when certain attributes cannot be inserted into the database without the presence of other attributes.

e_id	e_name	e_address	e_dept
101	Rick	Delhi	D001
101	Rick	Delhi	D002
123	Maggie	Agra	D890
166	Glenn	Chennai	D900
166	Glenn	Chennai	D004

- Suppose a new employee joins the company, who is under training and currently not assigned to any department then we would not be able to insert the data into the table if the **e_dept field doesn't allow nulls**.

Purpose of Normalization or schema refinement

- *Update anomaly*: An **Update Anomaly** exists when one or more Instances of duplicated data is updated, but not all.

e_id	e_name	e_address	e_dept
101	Rick	Delhi	D001
101	Rick	Delhi	D002
123	Maggie	Agra	D890
166	Glenn	Chennai	D900
166	Glenn	Chennai	D004

- In the above table, we have two rows for employee Rick as he belongs to two departments of the company. *If we want to update the address of Rick then we have to update the same in two rows or the data will become inconsistent.*

Purpose of Normalization or schema refinement

- **Delete anomaly:** A **Delete Anomaly** exists when certain attributes are lost because of the deletion of other attributes.

e_id	e_name	e_address	e_dept
101	Rick	Delhi	D001
101	Rick	Delhi	D002
123	Maggie	Agra	D890
166	Glenn	Chennai	D900
166	Glenn	Chennai	D004

- Suppose, if at a point of time the company closes the department D890 then deleting the rows that are having e_dept as D890 would also delete the information of employee Maggie.

UNIT IV: Schema Refinement (Normalization)

- Purpose of Normalization or schema refinement, **concept of functional dependency**, normal forms based on functional dependency Lossless join and dependency preserving decomposition, (1NF, 2NF and 3 NF), concept of surrogate key, Boyce-Codd normal form (BCNF), MVD, Fourth normal form (4NF), Fifth Normal Form (5NF).

Concept of functional dependency

- Introduction
- Armstrong Axioms
- Types of Functional Dependencies

Concept of functional dependency

- Introduction
- Armstrong Axioms
- Types of Functional Dependencies

Concept of functional dependency

- In DBMS, a functional dependency (FD) describes a relationship between attributes in a table where **the value of one attribute (or a set of attributes) uniquely determines the value of another attribute (or set of attributes)**.
- It's represented as **$X \rightarrow Y$** , where **X** is the **determinant** (the attribute that determines) and **Y** is the **dependent** (the attribute whose value is determined).

Concept of functional dependency

Reg_no	Name	Block	Dep_Name
42	abc	A	CSE
43	pqr	C	IT
44	xyz	B	ME
45	Xyz	C	AIDS

- **Reg_no** □ {Name}
- **Reg_no** □ {Name, Block, Dept_Name}
- **Block** □ {Dept_Name}

Concept of functional dependency

Employee number	Employee Name	Salary	City
1	Dana	50000	San Francisco
2	Francis	38000	London
3	Andrew	25000	Tokyo

- In this example, if we know the value of Employee number, we can obtain Employee Name, city, salary, etc. By this, we can say that the city, Employee Name, and salary are functionally depended on Employee number.

Concept of functional dependency

- Introduction
- Armstrong Axioms
- Types of Functional Dependencies

Armstrong's axioms

- Armstrong's Axioms is a set of rules.
- It provides a simple technique for reasoning about functional dependencies.
- It was developed by William W. Armstrong in 1974.
- It is used to infer all the functional dependencies on a relational database.
- Various Axioms Rules:
 - **A. Primary Rules**
 - **B. Secondary Rules**

Armstrong's axioms

□ Primary Rules:

Rule 1	Reflexivity: If A is a set of attributes and B is a subset of A, then A holds B. $\{ A \rightarrow B \}$
Rule 2	Augmentation: If A hold B and C is a set of attributes, then AC holds BC. $\{ AC \rightarrow BC \}$ It means that attribute in dependencies does not change the basic dependencies.
Rule 3	Transitivity: If A holds B and B holds C, then A holds C. If $\{ A \rightarrow B \}$ and $\{ B \rightarrow C \}$, then $\{ A \rightarrow C \}$

Armstrong's axioms

□ Secondary Rules:

Rule 1	Union If A holds B and A holds C, then A holds BC. If $\{A \rightarrow B\}$ and $\{A \rightarrow C\}$, then $\{A \rightarrow BC\}$
Rule 2	Decomposition If A holds BC and A holds B, then A holds C. If $\{A \rightarrow BC\}$ and $\{A \rightarrow B\}$, then $\{A \rightarrow C\}$
Rule 3	Pseudo Transitivity If A holds B and BC holds D, then AC holds D. If $\{A \rightarrow B\}$ and $\{BC \rightarrow D\}$, then $\{AC \rightarrow D\}$

Concept of functional dependency

- Introduction
- Armstrong Axioms
- Types of Functional Dependencies

Functional Dependencies-Types

- Trivial functional dependency
- Non-Trivial functional dependency
- Multivalued functional dependency
- Transitive functional dependency
- Partial dependency

Trivial functional dependency

- In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e. **If $X \rightarrow Y$ and Y is the subset of X .**

roll_no	name	age
42	abc	17
43	pqr	18

- Here, $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent **name is a subset of determinant set $\{\text{roll_no}, \text{name}\}$.**
- Similarly, **$\text{roll_no} \rightarrow \text{roll_no}$** is also an example of trivial functional dependency.

Functional Dependencies-Types

- Trivial functional dependency
- Non-Trivial functional dependency
- Multivalued functional dependency
- Transitive functional dependency
- Partial dependency

Non-Trivial functional dependency

- In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant.
i.e. If $X \rightarrow Y$ and **Y is not a subset of X** .

roll_no	name	age
42	abc	17
43	pqr	18

- Here, **$\text{roll_no} \rightarrow \text{name}$** is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll_no**.
- Similarly, **$\{\text{roll_no}, \text{name}\} \rightarrow \text{age}$** is also a non-trivial functional dependency, since **age** is **not a subset of $\{\text{roll_no}, \text{name}\}$**

Functional Dependencies-Types

- Trivial functional dependency
- Non-Trivial functional dependency
- Multivalued functional dependency
- Transitive functional dependency
- Partial dependency

Multivalued functional dependency

A table is said to have multi-valued dependency, if the following conditions are true,

- For a dependency $A \twoheadrightarrow B$, if for **a single value of A, multiple value of B exists**, then the table may have multi-valued dependency.
- Also, a table should have **at-least 3 columns** for it to have a multi-valued dependency.
- And, for a relation $R(A,B,C)$, if there is a multi-valued dependency between, A,B and C, then **B and C should be independent of each other.**

Multivalued functional dependency

Car_model	Maf_year	Color
H001	2017	Metallic
H001	2017	Green
H005	2018	Metallic
H005	2018	Blue
H010	2015	Metallic
H033	2012	Gray

- In this example, maf_year and color are independent of each other but dependent on car_model. In this example, these two columns are said to be multivalued dependent on car_model.
- This dependence can be represented like this:
- $\text{car_model} \twoheadrightarrow \text{maf_year}, \text{car_model} \twoheadrightarrow \text{colour}$

Functional Dependencies-Types

- Trivial functional dependency
- Non-Trivial functional dependency
- Multivalued functional dependency
- Transitive functional dependency
- Partial dependency

Transitive functional dependency

- In transitive functional dependency, **dependent is indirectly dependent on determinant**. i.e. If $a \rightarrow b$ & $b \rightarrow c$, then according to axiom of transitivity, $a \rightarrow c$.

enrol_no	name	dept	building_no
42	abc	CSBS	4
43	pqr	IT	2
44	xyz	CSE	1

- Here, **enrol_no $\rightarrow$ dept** and **dept $\rightarrow$ building_no**,
Hence, according to the axiom of transitivity, **enrol_no $\rightarrow$ building_no** is a valid functional dependency.
- This is an indirect functional dependency, hence called Transitive functional dependency.

Functional Dependencies-Types

- Trivial functional dependency
- Non-Trivial functional dependency
- Multivalued functional dependency
- Transitive functional dependency
- Partial dependency

Partial dependency

- Partial Dependency occurs when a non-prime attribute is functionally dependent on part of a candidate key.
- The 2nd Normal Form (2NF) eliminates the Partial Dependency.

StudentID	ProjectNo	StudentName	ProjectName
S01	199	Katie	Geo Location
S02	120	Ollie	Cluster Exploration

- Composite primary Key (StudentID, ProjectNo).
- The **StudentName** can be determined by **StudentID**, which makes the relation Partial Dependent.
- The **ProjectName** can be determined by **ProjectNo**, which makes the relation Partial Dependent.

UNIT IV: Schema Refinement (Normalization)

- Purpose of Normalization or schema refinement, concept of functional dependency, normal forms based on functional dependency Lossless join and dependency preserving decomposition, (1NF, 2NF and 3 NF), concept of surrogate key, Boyce-Codd normal form (BCNF), MVD, Fourth normal form (4NF), Fifth Normal Form (5NF).

Normal forms

- **Normalization** is the process of **minimizing redundancy from a relation or set of relations**. Redundancy in relation may cause insertion, deletion, and update anomalies.

The types of Normal Forms are:

- First Normal Form / 1NF
- Second Normal Form / 2NF
- Third Normal Form / 3NF
- Boyce Codd Normal Form (BCNF) / 3.5 NF
- Fourth Normal Form / 4NF
- Fifth Normal Form / 5NF

Normal forms

	1NF	2NF	3NF	4NF	5NF
Decomposition of Relation	R	R ₁₁ R ₁₂	R ₂₁ R ₂₂ R ₂₃	R ₃₁ R ₃₂ R ₃₃ R ₃₄	R ₄₁ R ₄₂ R ₄₃ R ₄₄ R ₄₅
Conditions	Eliminate Repeating Groups	Eliminate Partial Functional Dependency	Eliminate Transitive Dependency	Eliminate Multi-values Dependency	Eliminate Join Dependency

Normal forms

- First Normal Form / 1NF
- Second Normal Form / 2NF
- Third Normal Form / 3NF
- Boyce Codd Normal Form (BCNF) / 3.5 NF
- Fourth Normal Form / 4NF
- Fifth Normal Form / 5NF

First Normal Form / 1NF

- If a **relation contains a composite or multi-valued attribute**, it violates the first normal form.
- Or the relation is in first normal form if it does not contain any **composite** or **multi-valued attribute**.
- A relation is in first normal form if every attribute in that relation is singled valued attribute.

A table is in 1 NF if:

- There are only **Single Valued Attributes**.
- There is a **unique name for every Attribute/Column**.

First Normal Form / 1NF

- **Example-1:** Relation STUDENT in table 1 is not in 1NF because of multi-valued attribute STUD_PHONE. Its decomposition into 1NF has been shown in table 2.

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721, 9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 1



Conversion to first normal form

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721	HARYANA	INDIA
1	RAM	9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 2

Normal forms

- First Normal Form / 1NF
- Second Normal Form / 2NF
- Third Normal Form / 3NF
- Boyce Codd Normal Form (BCNF) / 3.5 NF
- Fourth Normal Form / 4NF
- Fifth Normal Form / 5NF

Second Normal Form / 2NF

- In the 2NF, relational **must be in 1NF**.
- In the second normal form, all **non-key attributes are fully functional dependent on the primary key** (No partial Dependencies).

Second Normal Form / 2NF

<StudentProject>

StudentID	ProjectID	StudentName	ProjectName
S89	P09	Olivia	Geo Location
S76	P07	Jacob	Cluster Exploration
S56	P03	Ava	IoT Devices
S92	P05	Alexandra	Cloud Deployment

Second Normal Form / 2NF

- The prime key attributes are **StudentID** and **ProjectID**.
- The **StudentName** can be determined by **StudentID**, which makes the relation Partial Dependent.
- The **ProjectName** can be determined by **ProjectID**, which makes the relation Partial Dependent.
- Therefore, the **<StudentProject>** relation violates the 2NF in Normalization and is considered a bad database design.

Second Normal Form / 2NF

<StudentInfo>

StudentID	ProjectID	StudentName
S89	P09	Olivia
S76	P07	Jacob
S56	P03	Ava
S92	P05	Alexandra

Second Normal Form / 2NF

<ProjectInfo>

ProjectID	ProjectName
P09	Geo Location
P07	Cluster Exploration
P03	IoT Devices
P05	Cloud Deployment

Normal forms

- First Normal Form / 1NF
- Second Normal Form / 2NF
- Third Normal Form / 3NF
- Boyce Codd Normal Form (BCNF) / 3.5 NF
- Fourth Normal Form / 4NF
- Fifth Normal Form / 5NF

Third Normal Form / 3NF

- A relation will be in 3NF if it is in 2NF and not contain any **transitive dependency**.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

Third Normal Form / 3NF

- Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key(EMP_ID). It violates the rule of third normal form.

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Third Normal Form / 3NF

□ **EMPLOYEE table:**

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

Third Normal Form / 3NF

□ **EMPLOYEE_ZIP table:**

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

Normal forms

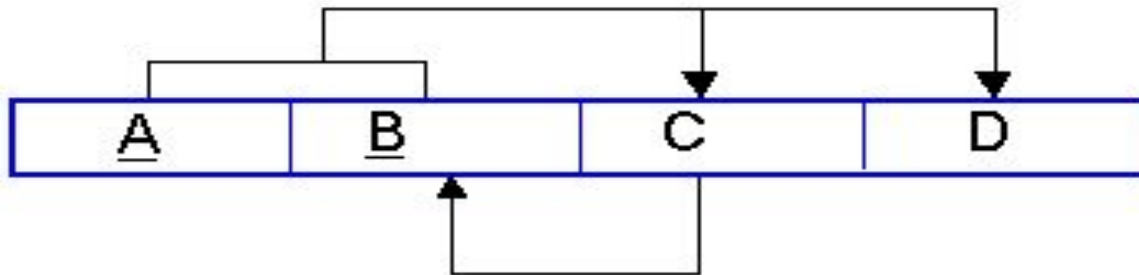
- First Normal Form / 1NF
- Second Normal Form / 2NF
- Third Normal Form / 3NF
- Boyce Codd Normal Form (BCNF) / 3.5 NF
- Fourth Normal Form / 4NF
- Fifth Normal Form / 5NF

Boyce Codd Normal Form (BCNF) / 3.5 NF

- It is an advance version of 3NF that's why it is also referred as 3.5NF.
- BCNF is stricter than 3NF.
- Rules:
 - It should be in the **Third Normal Form**.
 - And, for any dependency $A \rightarrow B$, A should be a **super key**.

Boyce Codd Normal Form (BCNF) / 3.5 NF

- The table is in 3NF. A and B are the keys and C and D depend on both A and B. There are no transitive dependencies existing between the non key attributes, C and D.
- The table is not in BCNF because a dependency exists between C and B. In other words if we know the value of C we can determine the value of B.



Boyce Codd Normal Form (BCNF) / 3.5 NF

□ **Example:**

<u>S_Num</u>	<u>T_Code</u>	Offering#	Review Date
123599	FIT104	01764	2nd March
123599	PIT305	01765	12th April
123599	PIT107	01789	2nd May
346700	FIT104	01764	3rd March
346700	PIT305	01765	7th May

- The dependencies are $S_Num, T_Code \twoheadrightarrow Offering\#, Review\ Date$ which means that the table is in third normal form.
- The table is not in BCNF as if we know the offering number we know who the teacher is. Each offering can only have one teacher!
 $Offering\# \twoheadrightarrow T_Code$ (A non key attribute is a determinant.)

Boyce Codd Normal Form (BCNF) / 3.5 NF

□ StudentReview

<u>S_Num</u>	<u>Offering#</u>	Review Date
123599	01764	2nd March
123599	01765	12th April
123599	01789	2nd May
346700	01764	3rd March
346700	01765	7th May

Boyce Codd Normal Form (BCNF) / 3.5

NF

□ OfferingTeacher

<u>Offering#</u>	T_Code
01764	FIT104
01765	PIT305
01789	PIT107

Normal forms

- First Normal Form / 1NF
- Second Normal Form / 2NF
- Third Normal Form / 3NF
- Boyce Codd Normal Form (BCNF) / 3.5 NF
- Fourth Normal Form / 4NF
- Fifth Normal Form / 5NF

Fourth Normal Form / 4NF

- It builds on the first three normal forms (1NF, 2NF and 3NF) and the Boyce-Codd Normal Form (BCNF).
- It states that, in addition to a database meeting the requirements of BCNF, it must **not contain multivalued dependency**.

Fourth Normal Form / 4NF

- Example:

STU_ID	COURSE	HOBBY
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

- The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity. Hence, there is no relationship between COURSE and HOBBY.
- In the STUDENT relation, a student with STU_ID, **21** contains two courses, **Computer** and **Math** and two hobbies, **Dancing** and **Singing**. So there is a Multi-valued dependency on STU_ID, which leads to unnecessary repetition of data.

Fourth Normal Form / 4NF

▣ STUDENT_COURSE

STU_ID	COURSE
21	Computer
21	Math
34	Chemistry
74	Biology
59	Physics

Fourth Normal Form / 4NF

▣ STUDENT_HOBBY

STU_ID	HOBBY
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

UNIT IV: Schema Refinement (Normalization)

- Purpose of Normalization or schema refinement, concept of functional dependency, normal forms based on functional dependency Lossless join and **dependency preserving decomposition**, (1NF, 2NF and 3 NF), concept of surrogate key, Boyce-Codd normal form(BCNF), MVD, Fourth normal form(4NF), Fifth Normal Form (5NF).

Dependency Preserving Decomposition

- A **dependency-preserving decomposition** is a process in relational database design where a table is split into smaller tables such that all the original functional dependencies (FDs) can still be checked and validated.
- In essence, every functional dependency from the original table either appears directly in one of the new smaller tables or can be logically derived from the FDs within the decomposed tables.
- *** *for example refer the note book.*

UNIT IV: Schema Refinement (Normalization)

- Purpose of Normalization or schema refinement, concept of functional dependency, normal forms based on functional dependency **Lossless join** and dependency preserving decomposition, (1NF, 2NF and 3 NF), concept of surrogate key, Boyce-Codd normal form (BCNF), MVD, Fourth normal form (4NF), Fifth Normal Form (5NF).

Lossless join & Lossy Join Decomposition

- In DBMS, a lossless decomposition is a process of decomposing a relation schema into multiple relations in such a way that it preserves the information contained in the original relation.
- In a lossless decomposition, the natural join of the decomposed relations must always produce exactly the original relation with no extra(spurious) or missing tuples.

Lossless join & Lossy Join Decomposition

- Determining Whether Decomposition Is Lossless Or Lossy:
- Consider a relation R is decomposed into two sub relations R_1 and R_2 .
- Then,
 - If all the following conditions satisfy, then the decomposition is lossless.
 - If any of these conditions fail, then the decomposition is lossy.

Lossless join & Lossy Join Decomposition

- **Condition-01:**

- Union of both the sub relations must contain all the attributes that are present in the original relation R.

- $R_1 \cup R_2 = R$

- **Condition-02:**

- Intersection of both the sub relations must not be null.

- In other words, there must be some common attribute which is present in both the sub relations

- $R_1 \cap R_2 \neq \emptyset$

Lossless join & **Lossy** Join Decomposition

□ **R:**

A	B	C
1	2	1
2	2	2
3	3	2

□ **R1(AB)**

A	B
1	2
2	2
3	3

R2(BC)

B	C
2	1
2	2
3	2

Lossless join & **Lossy** Join Decomposition

- Find the value of 'C' if the value of A is '1'?

A	B
1	2
2	2
3	3

B	C
2	1
2	2
3	2

A	B	B	C
1	2	2	1
1	2	2	2
1	2	3	2
2	2	2	1
2	2	2	2
2	2	3	2
3	3	2	1
3	3	2	2
3	3	3	2

Lossless join & **Lossy** Join Decomposition

- Find the value of 'C' if the value of A is '1'?

- Select R2.C from
R1 **Natural Join** R2
Where R1. A='1';

A	B	B	C
1	2	2	1
1	2	2	2
1	2	3	2
2	2	2	1
2	2	2	2
2	2	3	2
3	3	2	1
3	3	2	2
3	3	3	2

Lossless join & **Lossy** Join Decomposition

- Find the value of 'C' if the value of A is '1'?

- Select R2.C from
R1 **Natural Join** R2
Where R1. A='1';

- Natural Join Output:

A	B	B	C
1	2	2	1
1	2	2	2
1	2	3	2
2	2	2	1
2	2	2	2
2	2	3	2
3	3	2	1
3	3	2	2
3	3	3	2



Lossless join & **Lossy** Join Decomposition

- Find the value of 'C' if the value of A is '1'?

- Select R2.C from
R1 **Natural Join** R2
Where R1. A='1';

- Natural Join Output:

A	B	B	C
1	2	2	1
1	2	2	2
2	2	2	1
2	2	2	2
3	3	3	2

Lossless join & **Lossy** Join Decomposition

- Find the value of 'C' if the value of A is '1'?
- Select R2.C from R1 **Natural Join** R2 Where R1. A='1';

R

R'

□

A	B	C
1	2	1
2	2	2
3	3	2

A	B	B	C
1	2	2	1
1	2	2	2
2	2	2	1
2	2	2	2
3	3	3	2

Lossless join & **Lossy** Join Decomposition

- Find the value of 'C' if the value of A is '1'?
- Select R2.C from R1 **Natural Join** R2 Where R1. A='1';

R		
A	B	C
1	2	1
2	2	2
3	3	2

A	B	C
1	2	1
1	2	2
2	2	1
2	2	2
3	3	2

Lossless join & **Lossy** Join Decomposition

- Find the value of 'C' if the value of A is '1'?
- Select R2.C from R1 **Natural Join** R2 Where **R1.A='1'**;

R			R'		
A	B	C	A	B	C
1	2	1	1	2	1
2	2	2	1	2	2
3	3	2	2	2	1
			2	2	2
			3	3	2

Spurious
Tuples

Lossless join & Lossy Join Decomposition

□ R:

A	B	C
1	2	1
2	2	2
3	3	2

□ R1(AB)

A	B
1	2
2	2
3	3

R2(AC)

A	C
1	1
2	2
3	2

Lossless join & Lossy Join Decomposition

- Find the value of 'C' if the value of A is '1'?

A	B
1	2
2	2
3	3

A	C
1	1
2	2
3	2

A	B	A	C
1	2	1	1
1	2	2	2
1	2	3	2
2	2	1	1
2	2	2	2
2	2	3	2
3	3	1	1
3	3	2	2
3	3	3	2

Lossless join & Lossy Join Decomposition

- Find the value of 'C' if the value of A is '1'?

- Select R2.C from
R1 **Natural Join** R2
Where R1. A='1';

A	B	A	C
1	2	1	1
1	2	2	2
1	2	3	2
2	2	1	1
2	2	2	2
2	2	3	2
3	3	1	1
3	3	2	2
3	3	3	2

Lossless join & Lossy Join Decomposition

- Find the value of 'C' if the value of A is '1'?

- Select R2.C from
R1 **Natural Join** R2
Where R1. A='1';

A	B	A	C	
1	2	1	1	
1	2	2	2	×
1	2	3	2	×
2	2	1	1	×
2	2	2	2	
2	2	3	2	×
3	3	1	1	×
3	3	2	2	×
3	3	3	2	

Lossless join & Lossy Join Decomposition

- Find the value of 'C' if the value of A is '1'?

- Select R2.C from
R1 **Natural Join** R2
Where R1. A='1';

A	B	A	C
1	2	1	1
2	2	2	2
3	3	3	2

Lossless join & Lossy Join Decomposition

- Find the value of 'C' if the value of A is '1'?

- Select R2.C from
R1 **Natural Join** R2
Where R1. A='1';

A	B	A	C
1	2	1	1
2	2	2	2
3	3	3	2

Lossless join & Lossy Join Decomposition

- Find the value of 'C' if the value of A is '1'?
- Select R2.C from R1 **Natural Join** R2 Where R1. A='1';

R

R'

A	B	C		A	B	C
1	2	1	→	1	2	1
2	2	2	→	2	2	2
3	3	2	→	3	3	2

* *Always decompose the tables based on Candidate key / Primary Key **

Normal forms

- First Normal Form / 1NF
- Second Normal Form / 2NF
- Third Normal Form / 3NF
- Boyce Codd Normal Form (BCNF) / 3.5 NF
- Fourth Normal Form / 4NF
- Fifth Normal Form / 5NF

Fifth Normal Form / 5NF

- A relation is in 5NF if it is in 4NF and not contains any join dependency and **joining should be lossless**.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as **Project-join normal form (PJ/NF)**.

UNIT IV: Schema Refinement (Normalization)

- Purpose of Normalization or schema refinement, concept of functional dependency, normal forms based on functional dependency Lossless join and dependency preserving decomposition, (1NF, 2NF and 3 NF), **concept of surrogate key**, Boyce-Codd normal form (BCNF), MVD, Fourth normal form (4NF), Fifth Normal Form (5NF).

Concept of surrogate key

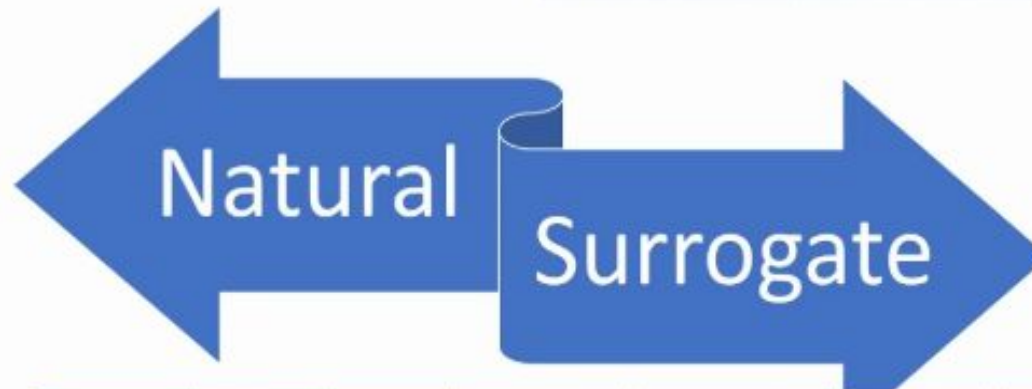
- A surrogate key in a database is a **system-generated, unique identifier** for each record that has no business context or meaning outside the database.
- Unlike natural keys (like an email address), which are derived from application data, **surrogate keys are artificial, typically integers, and are often used as the primary key to uniquely identify rows.**
- They provide stable, unchanging identifiers, simplify table design, improve performance, and are **essential when no natural key exists or when natural keys might change over time.**

Concept of surrogate key

SSN

Aadhar

EMP_ID	NAME	SSN
123	ABC	123-456-7980
124	XYZ	789-345-5678



ID	EMP_ID	NAME	Department	START_DATE	END_DATE	Flag
1	123	ABC	IT	1/17/2020	1/14/2021	N
2	124	XYZ	HR	1/18/2020	Null	Y
3	123	ABC	IT	1/15/2021	NULL	Y

Concept of surrogate key

SURROGATE Key in SQL

Key	Longitude	Latitude
KEY 1	220	140
KEY 2	240	170
KEY 3	220	160

Concept of surrogate key

Primary Key	Surrogate Key
A primary key uniquely identifies a row in a table.	A surrogate key is a special type of primary key that is system-generated and has no business meaning.
The key difference is that a primary key can be a "natural key" derived from existing data (like a Pan Number).	A surrogate key is an "artificial key" created by the database (like an auto-incrementing number)
Eg: RegdNo, Empno, Pan Number	Eg: Auto incremented Row No.